

Seven Seas A-18

Security Audit

Oct 16, 2024

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security audit was performed by the Macro security team from Oct 13th to Oct 14th 2024.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
High	1	-	-	1

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.

Source Code

The following source code was reviewed during the audit:

- **Repository:** [boring-vault](#)
- **Commit Hash:** `ec74b24330a1d7b144dc397c1c20c76e3d6fc460`

Specifically, we audited the changes for the following contracts in [PR 139](#) as well as a review of the safety of integrating with usual money contracts.

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/Protocols/UsualMoneyDecoderAndSanitizer.sol</code>	<code>66c6bc3ad3909f7c5eb87600d96b34c5db2ecabace8ceb722fdbebabade465f1</code>
<code>src/base/DecodersAndSanitizers/EtherFiLiquidityDecoderAndSanitizer.sol</code>	<code>54a1139d43210eee0c64c60172fbd9ea45fb6e22b4ccacb01550897cd15fdb9c</code>

Additionally, the SatLayer protocol was reviewed and its associated decoder from [PR 135](#) was audited.

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/Protocols/SatlayerStakingDecoderAndSanitizer.sol</code>	<code>d9c6eb28f54a0b0663b24500183b64abfa5b3e951560172b33abd11c2e8ed6e8</code>
<code>src/base/DecodersAndSanitizers/PointFarmingDecoderAndSanitizer.sol</code>	<code>119466decbbcb9a9cbf77822ff91ed1b5699f2e2c44bdd214c0865ace188d5ebd</code>

Additionally, the Bitcorn protocol was reviewed and its associated staking decoder from [PR 134](#) was audited.

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/CornStakingDecoderAndSanitizer.sol	7f8ad1b46dd44e04aa9045cbe541c6d4035e2fef68b153780469870043efc0d6
src/base/DecodersAndSanitizers/PointFarmingDecoderAndSanitizer.sol	119466decbbcb9a9cbf77822ff91ed1b5699f2e2c44bdd214c0865ace188d5ebd

Additionally, the Frax staking was reviewed and its associated staking decoder from [PR 127](#) was audited.

Additionally, the BoringVault Withdrawal Queue and Solver from [PR 130](#) was audited.

Source Code	SHA256
src/base/Roles/BoringQueue/BoringOnChainQueue.sol	22cb9ca512a20abad78c4003085b3206ce3931c364eb2132f7e431e76f8fc13b
src/base/Roles/BoringQueue/BoringOnChainQueueWithTracking.sol	5ec66a43fae691e62bafc816951bb78bb91c604d9417f8598fd93427f2addd53
src/base/Roles/BoringQueue/BoringSolver.sol	e304f4a60e1b66f9acf8ba2b5546ea38b4082832ef4f77b0ed6946613e97b4ca
src/base/Roles/BoringQueue/IBoringSolver.sol	a4fcf8547590d051a745fddac0f003364f49b3364028694eb8cd46b1a7a0a3dd

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/FraxDecoderAndSanitizer.sol	153a24b13e96d3f3826196e40de79bfa8183615daf75a4a06fb95a7855d24560
src/base/DecodersAndSanitizers/StakingDecoderAndSanitizer.sol	2807965e2b0341ebc401f03a1c73da0593058fa29ec0894368adb57601700364

Additionally, Lido wstETH bridging was reviewed and its associated bridging decoder from [PR 126](#) was audited.

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/LidoStandardBridgeDecoderAndSanitizer.sol	3f2d9547ccdd9f096393f4dfa7eb1091b213dae9d4f4fdeba4dc141ba46d56d4
src/base/DecodersAndSanitizers/BridgingDecoderAndSanitizer.sol	1c0619fc811d6eb2472c7b5c39f70b4e9ab4d4610acb9e5ee65e9629a0a2f5b8

Additionally, we reviewed the use of Elixir withdrawals using the Ethena decoder defined in [PR 133](#) as well as a review of the safety of integrating with usual money contracts.

Eigen Layer delegation and claiming functionality was added and reviewed.

Source Code	SHA256
src/base/DecodersAndSanitizers/Protocols/EigenLayerLSTStakingDecoderAndSanitizer.sol	cef6745353dd2e407961006832415ef4f2d8e7533a7c8b6a43ba4ce759a6af07

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.



Users can bypass withdrawal queue

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

H-4

Users can bypass withdrawal queue

TOPIC

Protocol Design

STATUS

Fixed [↗](#)

IMPACT

High

LIKELIHOOD

Medium

`BoringOnChainQueue.sol` allows users to request to withdraw BoringVault shares for accepted withdrawal assets, although there is a waiting period before the request can be fulfilled. Requests are intended to be fulfilled by users or solvers via the `solveOnChainWithdraws()` function.

The `BoringSolver.sol` contract can fulfill more complex solves and has permission to make `bulkWithdrawal()` and `bulkDeposit()` calls on Boring Vaults which allows them to fulfill withdrawals for users in the withdrawal queue.

The execution path for the BoringSolver is to call an initial external function it initiate the required solve type, like `boringRedeemMintSolve()`, which then calls `solveOnChainWithdrawals` on the queue with the required `solveData`, with the `BoringOnChainQueue` calling back `boringSolve()` to complete the solve using the initially passed in `solveData`.

However, `boringSolve()` is an external function that has no explicit permission guard, and assumes the caller is the queue:

```
function boringSolve(
    address initiator,
    address boringVault,
    address solveAsset,
    uint256 totalShares,
    uint256 requiredAssets,
```

```

    bytes calldata solveData
) external requiresAuth {
    if (initiator != address(this)) revert BoringSolver___WrongInitiator();

    address queue = msg.sender;

    SolveType solveType = abi.decode(solveData, (SolveType));

    if (solveType == SolveType.BORING_REDEEM) {
        _boringRedeemSolve(queue, solveData, boringVault, solveAsset, totalShare);
    } else if (solveType == SolveType.BORING_REDEEM_MINT) {
        _boringRedeemMintSolve(queue, solveData, boringVault, solveAsset, totalShare);
    } else {
        // Added for future protection, if another enum is added, txs with that type will fail
        // if no changes are made here.
        revert BoringSolver___FailedToSolve();
    }
}

```

Reference: [BoringSolver.sol#L121-L147](#)

Any contract/eoa can pass the `initiator` as the BoringSolver contract address, send into the contract the desired boringVault shares and make use of its permission to withdraw via either solve type and get around the withdrawal delay. Users could leverage this and frontrun/backrun share price updates to immediately benefit without being required to hold onto shares long term.

Remediations to Consider

Require the caller of `boringSolve()` to be a trusted withdrawal queue to prevent any user from taking advantage of `BoringSolver` permissions.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.